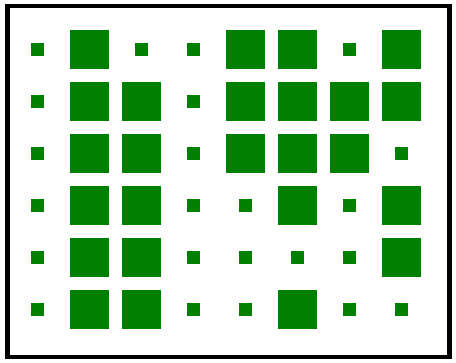
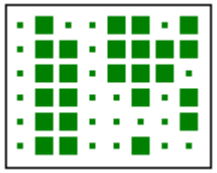




Class 12
Deep C Diving: Strings, Preprocessor, and Pointers

CS 330
Programming Languages
And Compilers

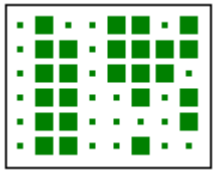
Obtain the Lab 4 Starting Code



- Open a terminal and change directory into your **cs330-IndividualLabWork** directory
- Use git to obtain the lab code:

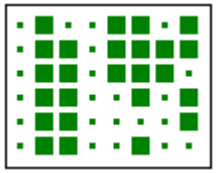
```
git pull
```

- The directory **lab4-C-ArraysAndPointers** should be added to your project
  - It contains several example C programs which we will explore
- You will create additional C programs in this directory for your individual lab assignment



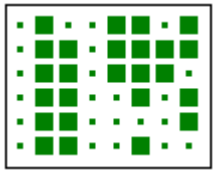
C Character Arrays

No Strings; Character Arrays



- C has no data type for strings
  - Character arrays are used to mimic strings
- C doesn't range check arrays so there needs to be a flag at the end of the "string"
  - C libraries expect a 0 byte to signal the end of the "string"
    - **Null terminator**
    - Escape sequence is '\0'

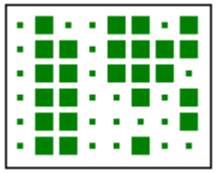
String Literals Become Char. Arrays



- String literals are enclosed in double quotes
  - The compiler adds the null terminator to string literals
- For example, for the string literal “hello”, the compiler creates a character array containing six elements

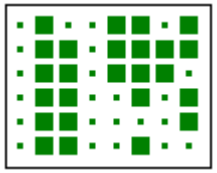
h	e	l	l	o	\0

Missing ‘\0’ Is a Common Issue



- Manipulating character data in programs is very common
- Forgetting to include the null terminator will cause a variety of issues, such as accessing elements beyond the bounds of an array
- If you are managing character array data, pay attention to how you are ensuring that you never go beyond the bounds of your character array and that you have accounted for, and inserted, the null terminator

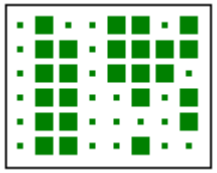
Longest Line (1/3)



- A program to read input lines and output only the longest line
  - Requires storing the current longest in a character array
- File: **`longestline.c`** (over 3 slides)

```
#include <stdio.h>
#define MAXLINE 1000    /* maximum input line length */
int getnextline(char line[], int maxline);
void copy(char to[], char from[]);
/* print longest line */
int main() {
        int len;        // current line length
        int max;        // maximum length seen so far
        char line[MAXLINE];     // current input line
        char longest[MAXLINE];  // longest line found so far
        max = 0;
```

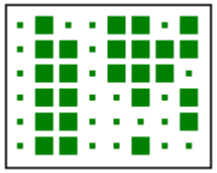
Longest Line (2/3)



```
while ((len = getnextline(line, MAXLINE)) > 0)
        if (len > max) {
                max = len;
                copy(longest, line);
        }
    if (max > 0) // there was a line
        printf("%s\n", longest);
    return 0;
}

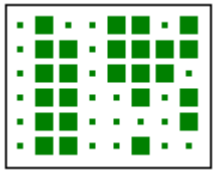
/* getnextline: read a line into s, return length of line */
int getnextline(char s[], int lim) {
    int c, i;
    for (i = 0;i < lim - 1 && (c = getchar()) != EOF && c != '\n';++i)
        s[i] = c;
    if (c == '\n') {
        s[i] = c;
        ++i;
    }
    s[i] = '\0';
    return i;
}
```

Longest Line (3/3)



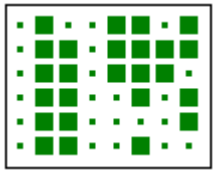
```
/* copy: copy 'from' into 'to': assume it is big enough */
void copy(char to[], char from[]) {
        int i;
        i = 0;
        while ((to[i] = from[i]) != '\0')
                ++i;
}
```

- Anything bother you about this program's design
- Is it guaranteed to find the longest line in a file? Why or why not?



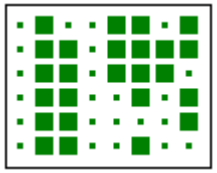
External Variables and Scope

C Variable Scope: Local/Automatic



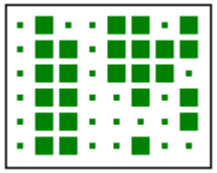
- So far, our variables have been declared within a function or as parameters to a function
  - Known as **local** or **automatic** variables
  - Private to the function; no other function has direct access to them
  - Created when the function is called and removed when the function exits
    - Contain “garbage” until initialized

C Variable Scope: External/Global



- Global variables are defined outside any function and available to all functions
  - Exist permanently during program execution
- Must be defined once, but declared in each source file using it
  - The **`extern`** keyword redeclares the variable without redefining it
    - Declare → state it exists
    - Define → declare and allocate memory

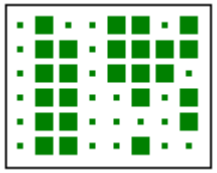
Longest Line v2 (1/3)



- We'll look at an updated longest line program which uses global variables instead of passing parameters
  - Global variables should be avoided whenever possible: wasted memory, concurrency issues, etc. Only doing this to show global variables in use
- Files:

```
longestline2.h
longestline2.c
longestline_vars.c
```

Longest Line v2: longestline2.h



```
// Header file for longestline2 example

#ifndef LONGESTLINE2H

#define LONGESTLINE2H

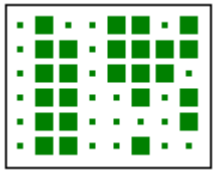
#define MAXLINE 1000  /* maximum input line length */

extern char line[MAXLINE];  // current input line
extern int max;      // maximum length seen so far
extern char longest[MAXLINE]; // longest line found so far

int getnextline(void);
void copy(void);

#endif // LONGESTLINE2H
```

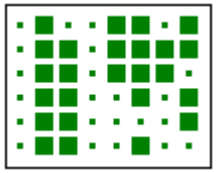
Longest Line v2: longestline2_vars.c



```
#include "longestline2.h"

// Define the global variables - allocate memory storage
int max;       // maximum length seen so far
char line[MAXLINE];  // current input line
char longest[MAXLINE];  // longest line found so far
```

Longest Line v2: longestline2.c (1/2)



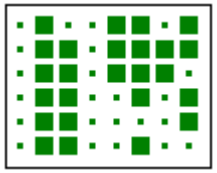
```
#include <stdio.h>
#include "longestline2.h"

/* print longest line */
int main() {
    int len;    // current line length

    max = 0;
    while ((len = getnextline()) > 0)
        if (len > max) {
            max = len;
            copy();
        }
    if (max > 0) // there was a line
    printf("%s\n", longest);

    return 0;
}
```

Longest Line v2: longestline2.c (2/2)



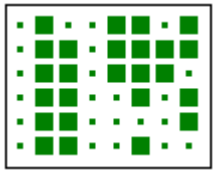
```
/* getnextline: specialized version using external array */
int getnextline(void) {
    int c, i;

    for (i = 0;i < MAXLINE - 1 && (c = getchar()) != EOF && c != '\n';++i)
        line[i] = c;
    if (c == '\n') {
        line[i] = c;
        ++i;
    }
    line[i] = '\0';
    return i;
}

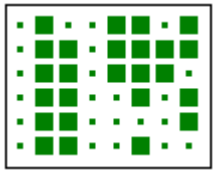

/* copy: specialized version using external array */
void copy() {
    int i;

    i = 0;
    while ((longest[i] = line[i]) != '\0')
        ++i;
}
```

Longest Line v2 Discussion

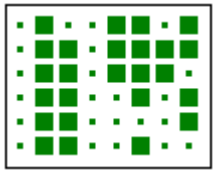


- What are some pros and cons of using global variables versus passing parameters?



C: Preprocessor

Preprocessor Introduction: #include



- We've seen two preprocessor directives

  `#include`

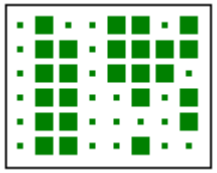
  - Includes the contents of another file at the location where the #include directive is found
  - Angle brackets <> used for files found in the standard libraries directory

    ```
    #include <stdio.h>
    ```

  - Quotes used for files in a local directory

    ```
    #include "myheader.h"
    ```

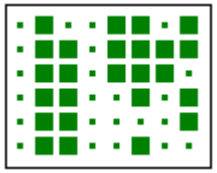
Preprocessor Introduction: #define



`#define`

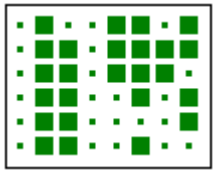
- Used for macro substitution
- We've used it for simple text replacement, but it can be used to create more interesting substitutions

Macro Substitution (1/2)



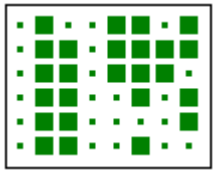
- Scope of macro is from definition to end of the source code file (or explicit undefining of macro)
- Macro may use previously defined macros
- Macro replacement *does not occur* within quoted strings
- Macros may be defined with arguments, which, *deceptively*, make them look like function calls
  - Unlike function calls, a macro expands inline
  - Must be careful of side effects in expansion

Macro Substitution (2/2)



- Remember, macros are NOT statements and do not end with semicolons
  - Adding a semicolon makes the semicolon part of the replacement text of the macro

A Macro With Arguments



- Macro for finding the maximum of two values

```
#define max(A, B) A > B ? A : B
```

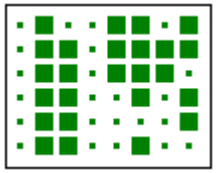
- Later in the code, we include the statement:

```
x = max(p, r);
```

- Preprocessor updates statement as:

```
x = p > r  ? p : r;
```

Unintended Side-Effects with Macros



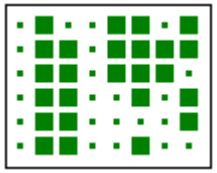
- Macros can lead to unexpected problems if they are treated like functions. Example:

```
#define max(A, B) A > B ? A : B
x = max(i++, j++);
```

- Or:

```
#define square(x) x * x
y = square(z + 1)
```

Additional Macro Details: #undef

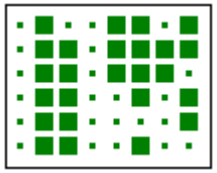


- A macro can be undefined (removed) by using **`#undef`**
- For example

```
#undef max
```

  - Undefines the macro for `max` from this point forward in the source file

Additional Macro Details: *Arg Name*



- *#ARG_NAME* in a macro concatenates the argument literal in a string
- For example, given the macro definition:

```
#define dprint(e) printf(#e " = %g\n", e)
```

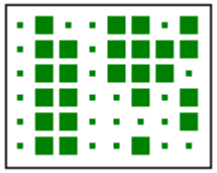
- And given the statement:

```
dprint(x / y);
```

- Preprocessor would update the statement to:

```
printf("x / y = %g\n", x / y);
```

Macro Test Program

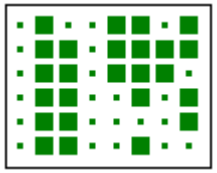


- We'll try out a few macros
- File: **macrotest.c**

```
#include <stdio.h>
#define max(A, B) A > B ? A : B
#define square(x) x * x
#define dprint(e) printf(#e " = %g\n", e)
int main() {
        int x;
        int a = 2;
        int b = 3;
        x = max(a, b);
        printf("%d\n", x);
        x = max(a++, b++);
        printf("%d %d %d\n", x, a, b);
        x = square(a + 1);
        printf("%d %d\n", a + 1, x);
        dprint(5.0 / 2.0);
}
```

- How can we fix the **square** macro? Is there a best practice you would follow for safer macros?

Optional Code (1/2)



- The C preprocessor may also be used to control optional inclusion of code
  - Useful when program includes platform-specific code, e.g., supporting different CPUs

`#ifdef` *`IDENTIFIER`*

  - Include code if *`IDENTIFIER`* is defined

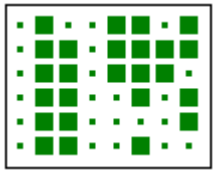
`#ifndef` *`IDENTIFIER`*

  - Include code if *`IDENTIFIER`* is not defined

`#endif`

  - Ends optional code block

Optional Code (2/2)



#if *EXPRESSION*

- Include code if *EXPRESSION* **is non-zero**

#elif EXPRESSION

- Include code if prior #if was false and *EXPRESSION* **is non-zero**

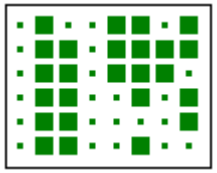
#else

- Include code if #if and all #elif were false

#endif

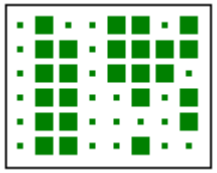
- Ends the optional code range in the source

Optional Code Example



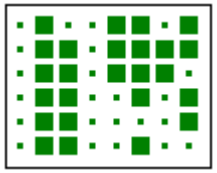
- We'll try out the #ifdef... preprocessor directives
- File: **optional.c**

```
#include <stdio.h>
/* Use the optional code preprocessor feature */
#define SET1 0
#define SET2 0
int main() {
        int x, y;
#ifdef SET1
        x = 1;
#else
        x = 2;
#endif
#if SET2
        y = 1;
#elif SET1
        y = 2;
#else
        y = 3;
#endif
        printf("x=%d\ty=%d\n", x, y);
}
```



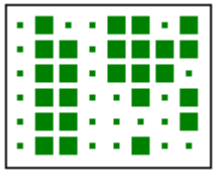
C: Pointers and Arrays

Pointer and Array Overview



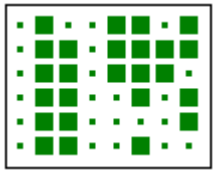
- A pointer **is a variable** that **contains** the **address** of another variable
  - Pointer with value 0 represents NULL
- Pointers and arrays are related
  - Array variable is a label for the first element of the array (e.g., a reference to the first element)

Pointer and Array Overview



- Used idiomatically, pointers make code clear, simple, and efficient
- Consider the basic organization of memory in a computer: consecutively numbered bytes
  - The number of bytes used for a variable depends on the data type's size
    - To properly decode a value in memory we need its starting location and size in bytes

Pointer Example



- `char` variable `c` and pointer to char variable `p`

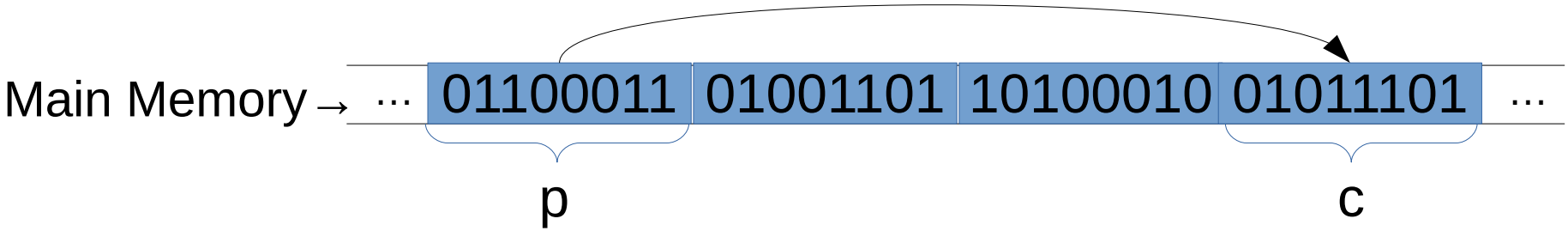
```
char c;
char *p;
```

- Unary `&` gets the address of a variable:

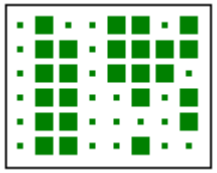
```
p = &c;
```

- This assigns address of `c` to the variable `p`

  p "points to" c

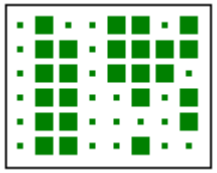


Pointer Example



- Unary * is the **indirection** or **dereferencing** operator
  - When used with a pointer, it accesses the value in the location being pointed to
    - “dereferences the pointer”

Pointer Example

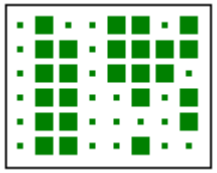


- Using `p` and `c` from the earlier slide, we can change the value in `c` by dereferencing `p`:

```
char c;
char *p;
p = &c;
*p = 15;
```

- Assign 15 to the location pointed to by `p`
  - Since *p points to c's memory*, the assignment updates `c`'s value
  - `c` is 15

Basic Pointer Usage (1/2)

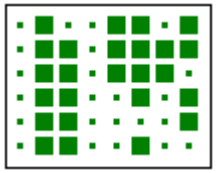


- We'll review code with some simple pointer operations
- File: **pointer1.c** (two pages)

```
#include <stdio.h>
int main() {
 int x = 1, y = 2, z[6];
 int *ip;
 for (int i = 0;i < 6;++i)
     z[i] = 0;
 // Display the initial values and pointer/label addresses
 printf("After initialization:\n");
 printf("z=%p &z=%p *z=%d &z[0]=%p\n", z, &z, *z, &z[0]);
 printf("x=%d\ty=%d\tip=%p\n", x, y, ip);
 printf("*ip=%d\n", *ip);
 for (int i = 0;i < 6;++i)
     printf("z[%d]=%d\t", i, z[i]);
 printf("\n");
```

Program may crash here!

Basic Pointer Usage (2/2)

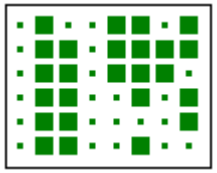


```
        // Manipulate the values using pointers
        ip = &x;
        printf("ip=%p &x=%p &ip=%p *ip=%d\n", ip, &x, &ip, *ip);
        y = *ip;
        *ip = 0;
        ip = &z[4];
        *ip = 25;

        // Display the resulting values
        printf("\nAfter manipulation\n");
        printf("x=%d\ty=%d\t*ip=%d\tip=%p\n", x, y, *ip, ip);
        for (int i = 0;i < 6;++i)
                printf("z[%d]=%d\t", i, z[i]);
        printf("\n");
}
```

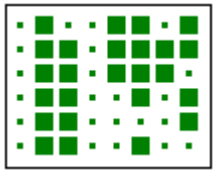
- Make sure you understand how each of the values being shown is set and what it represents (e.g. program data versus memory address)

Pointers and Function Arguments



- C passes arguments by value
  - Function cannot modify a variable defined in the calling function
- Using pointers we have a way for a function to modify a variable in the caller via indirection (pointers)
  - If the caller passes the address of its variable, the called function can change the value in the caller's variable (call by reference)

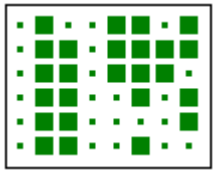
Sort Version 1



- Here is a simple program which tries to sort some ints
- File: **sort1.c**

```
#include <stdio.h>
/* Simple sorter version 1 */
void swap(int a, int b);
int main() {
	int vals[] = { 3, 2, 1 };
	for (int i = 0;i < 2;i++)
		for (int j = 0;j < 2;j++)
			if (vals[j] > vals[j + 1])
				swap(vals[j], vals[j + 1]);
	for (int i = 0;i < 3;++i)
		printf("%d\t", vals[i]);
	printf("\n");
}
/* Swap two ints - well, try to */
void swap(int a, int b) {
	int temp = a;
	a = b;
	b = temp;
}
```

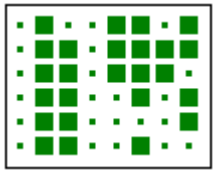
Sort Version 1



- Here we use pointers so the `swap()` works, goal is sorting ints
- File: **sort2.c**

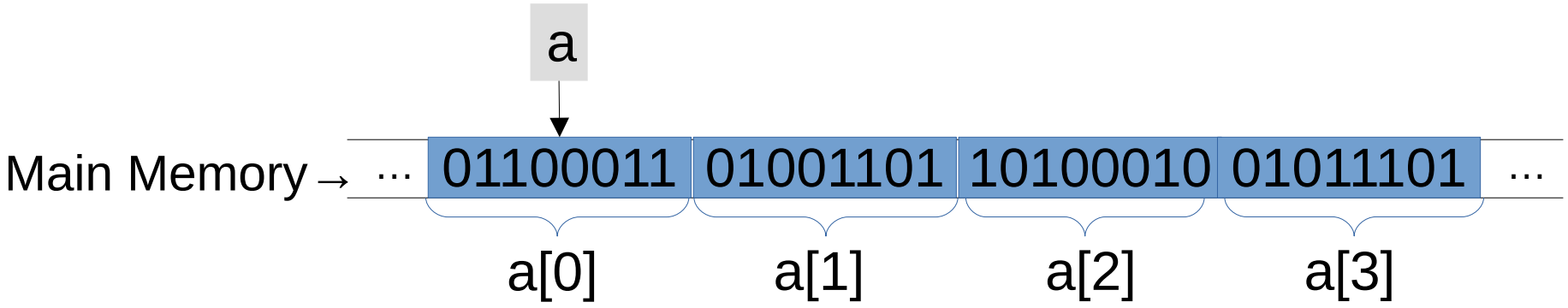
```
#include <stdio.h>
/* Simple sorter version 2 */
void swap(int *a, int *b);
int main() {
	int vals[] = { 3, 2, 1 };
	for (int i = 0;i < 2;i++)
		for (int j = 0;j < 2;j++)
			if (vals[j] > vals[j + 1])
				swap(&vals[j], &vals[j + 1]);
	for (int i = 0;i < 3;++i)
		printf("%d\t", vals[i]);
	printf("\n");
}
/* Swap two ints using pointers (memory addresses) */
void swap(int *a, int *b) {
	int temp = *a;
	*a = *b;
	*b = temp;
}
```

Pointers and Arrays

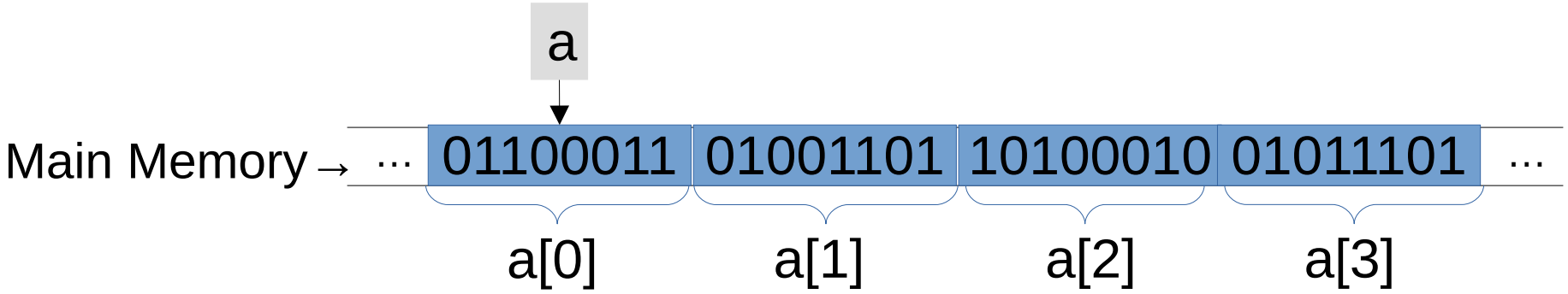


- Pointers can be used for array subscripting
  - Remember, array variable is a label for the first element (the element's address)

```
int a[4];
```

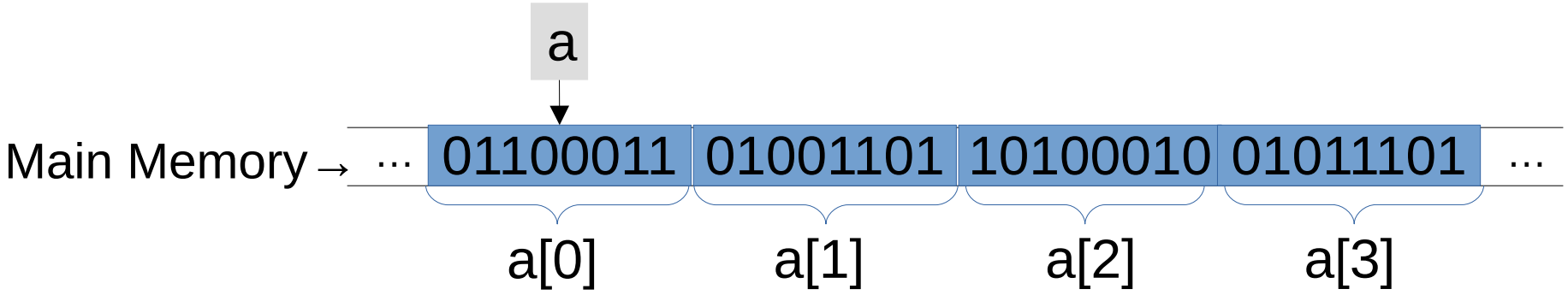


Pointers and Arrays



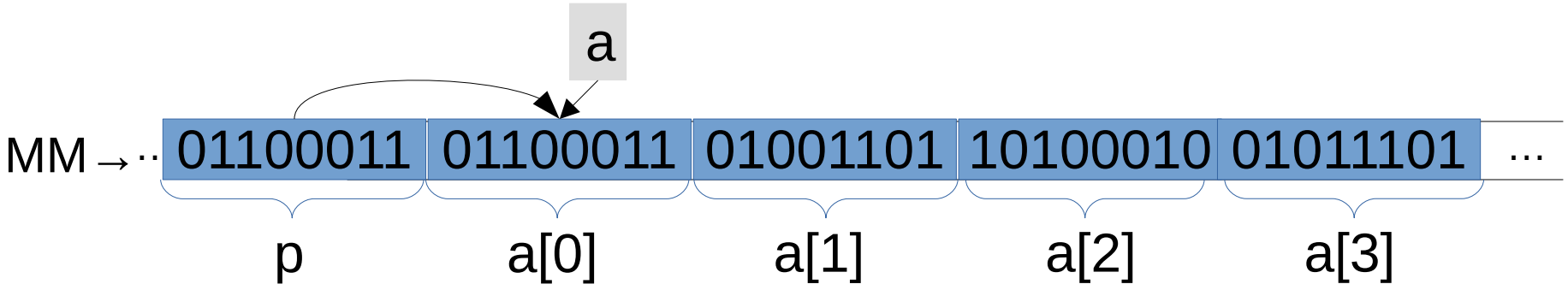
- Array elements are allocated in sequential memory locations
  - If **`a`** is the address of **`a[0]`** and **`x`** is an integer, then **`a + x`** is the address of element **`a[x]`**

Pointers and Arrays



- Data types may use different amounts of memory (char is 2 bytes, int is often 4 bytes)
  - Adding an integer value to a pointer results in “pointer math”
  - C knows the sizes of different data types and adds **`x * sizeof(type)`** to the pointer’s address value

Pointers and Arrays

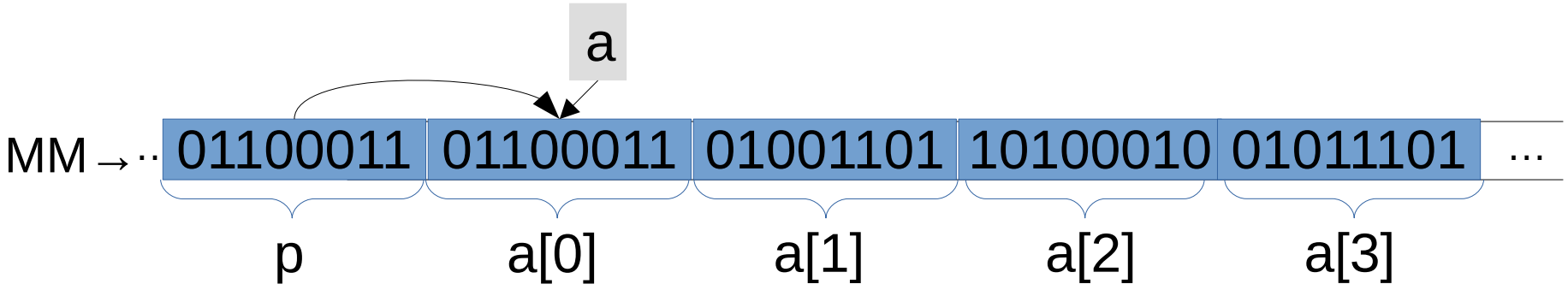


- Consider several equivalencies

```
int *p;
int a[4];
p = a; // is legal, both represent int pointers
```

- `a`, `p`, and `&a[0]` are the 1st element's address
- `a[0]`, `*a`, and `*p` are first element's value
- `&a[1]`, `a+1`, and `p+1` are the 2nd element's address
- `a[1]`, `*(a+1)`, and `*(p+1)` are the 2nd element's value

Pointers and Arrays

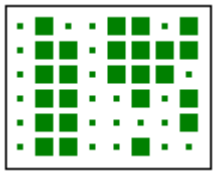



```
int a[4];
int *p = a;
```

- Fundamental difference between `a` and `p`
  - `a` is an array (label); value (address of 1st element) cannot be changed (`a++` is illegal)
  - `p` is a pointer; may change its value

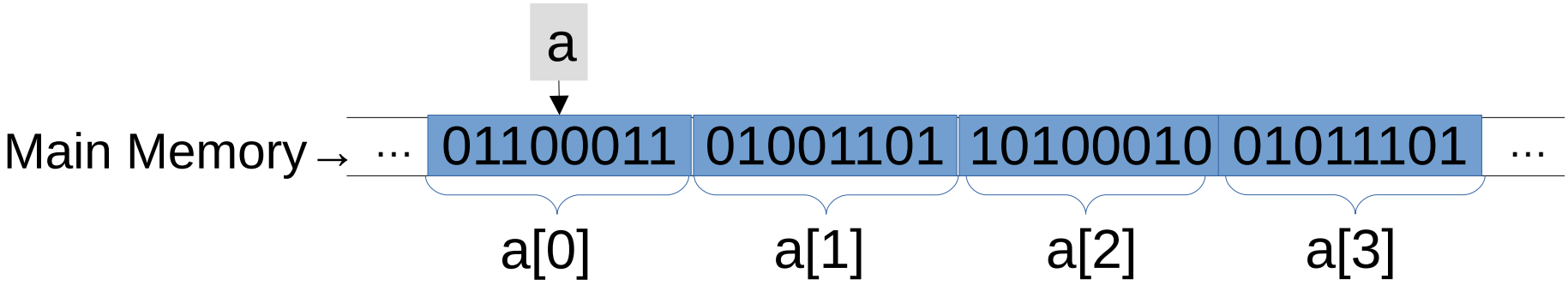
    `p++;` is legal, updates `p` to point to the next element in the array (`a[1]`)

Address Arithmetic

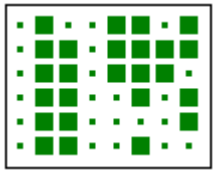


May compare addresses that are part of the same array (results are meaningful)

- Example: &a[2] - &a[0] will be 2
  &a[2] > &a[1] will be 1 (true)
- Edge case uses address of the first element beyond the array in calculations
  - For example &a[4] - &a[0] will be 4
    - The size of the array



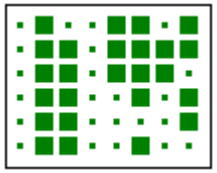
Practicing with Pointers (1/4)



- We'll review several versions of a small program to count the number of characters in a character array
- Our goal is to become familiar with common pointer idioms in C
- File: **charcpy1.c**

```
#include <stdio.h>
#include <string.h>
/* simple character array copy version 1 */
#define THE_MESSAGE "To be or not to be"
int scpy(char *s, char *t);
int main() {
        char cpy[strlen(THE_MESSAGE) + 1];
        *cpy = '\0';
        printf("cpy=%s\n", cpy);
        scpy(cpy, THE_MESSAGE);
        printf("cpy=%s\n", cpy);
}
/* scpy: copy values in array t to array s */
int scpy(char *s, char *t) {
        int i = 0;
        while ((s[i] = t[i]) != '\0')
                i++;
}
```

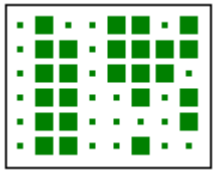
Practicing with Pointers (2/4)



- File: **charcpy2.c**
- Describe what was changed in scpy() from version 1

```
#include <stdio.h>
#include <string.h>
/* simple character array copy version 2 */
#define THE_MESSAGE "To be or not to be"
int scpy(char *s, char *t);
int main() {
        char cpy[strlen(THE_MESSAGE) + 1];
        *cpy = '\0';
        printf("cpy=%s\n", cpy);
        scpy(cpy, THE_MESSAGE);
        printf("cpy=%s\n", cpy);
}
/* scpy: copy values in array t to array s */
int scpy(char *s, char *t) {
        while ((*s = *t) != '\0') {
                s++;
                t++;
        }
}
```

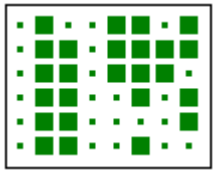
Practicing with Pointers (3/4)



- File: **charcpy3.c**
- Describe what was changed in scpy() from version 2

```
#include <stdio.h>
#include <string.h>
/* simple character array copy version 3 */
#define THE_MESSAGE "To be or not to be"
int scpy(char *s, char *t);
int main() {
        char cpy[strlen(THE_MESSAGE) + 1];
        *cpy = '\0';
        printf("cpy=%s\n", cpy);
        scpy(cpy, THE_MESSAGE);
        printf("cpy=%s\n", cpy);
}
/* scpy: copy values in array t to array s */
int scpy(char *s, char *t) {
        while ((*s++ = *t++) != '\0')
                ;
}
```

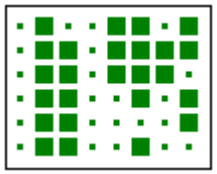
Practicing with Pointers (4/4)



- File: **charcpy4.c**
- Describe what was changed in scpy() from version 3

```
#include <stdio.h>
#include <string.h>
/* simple character array copy version 4 */
#define THE_MESSAGE "To be or not to be"
int scpy(char *s, char *t);
int main() {
        char cpy[strlen(THE_MESSAGE) + 1];
        *cpy = '\0';
        printf("cpy=%s\n", cpy);
        scpy(cpy, THE_MESSAGE);
        printf("cpy=%s\n", cpy);
}
/* scpy: copy values in array t to array s */
int scpy(char *s, char *t) {
        while (*s++ = *t++)
                ;
}
```

Multi-dimensional Array Syntax



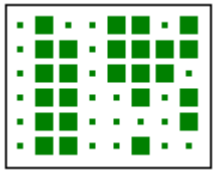
- Declared with pairs of square brackets

```
int scores[2][9];
```

  - Creates a two dimensional array with 2 rows and 9 columns

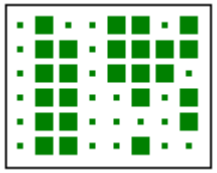
```
scores[0][0] = 5; // assigns 5 to 1st row/col
scores[0][5] = 10; // assigns 10 to 1st row, 6th col
scores[1][9] = 3; // assigns 3 to 2nd row, 10th col
```

Arrays of Pointers



- Creating pointer arrays
  - Pointers to pointers
  - Also called **vectors** in C
- Similar to statically defined multi-dimensional arrays with two significant differences:
  - Dimensions in a statically defined multi-dimensional array are fixed size (rectangular)
  - Using an array of pointers, each pointer may point to a single value or to an array, and those arrays may be of different sizes (jagged)

Example Array of Character Pointers

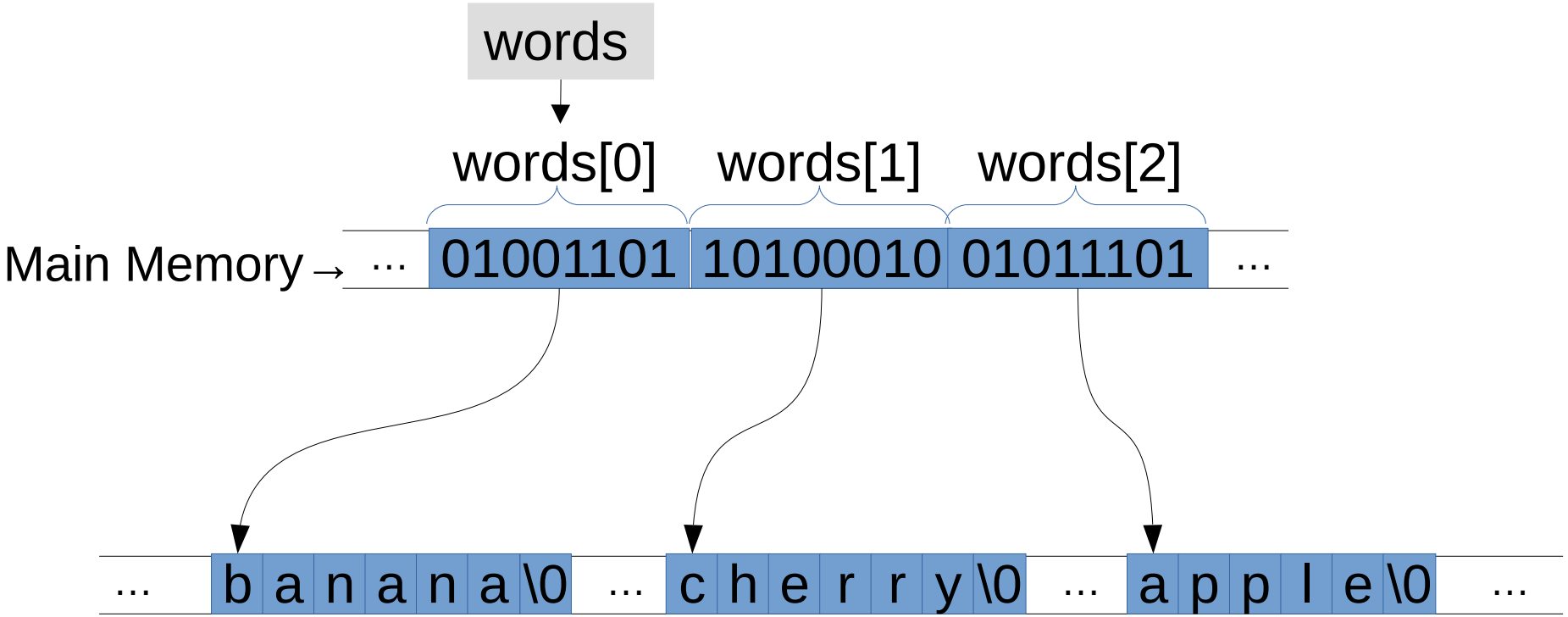


- Consider array of character pointers

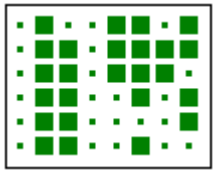
```
char *words[3];
words[0] = “banana”;
words[1] = “cherry”;
words[2] = “apple”;
```

  - Array named **words** with 3 elements, each holds a pointer to a character array
  - Three character arrays, each ending with a null terminator (e.g. ‘\0’)

Example Array of Character Pointers



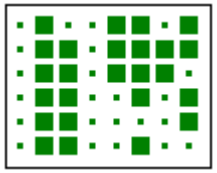
Iterate Through Array of Pointers (1/2)



- We'll iterate through the array of arrays
- File: **pointerarray.c** (two pages)

```
#include <stdio.h>
/* Iterate through a pointer array of character arrays */
int slen(char *s);
int main() {
        char *words[3];
        words[0] = "banana";
        words[1] = "cherry";
        words[2] = "apple";
        // print out the arrays
        for (int i = 0;i < 3;++i) {
                char *letter = words[i];
                printf("length %d\t", slen(words[i]));
                while (*letter)
                        printf("[%c]", *letter++);
                printf(" \n");
        }
}
```

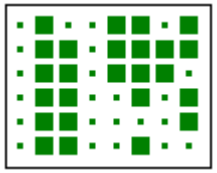
Iterate Through Array of Pointers (2/2)



```
/* Calculate the length of the string, not counting the null terminator.
   This is provided to see how we can use pointer arithmetic to
   traverse an array. The function strlen() should be used to
   obtain the length of a string. */
int slen(char *s) {
     char* e = s;
     while (*e++)
          ;
     return e - s;
}
```

- Look carefully through the code to make sure you understand all the pointer and value referencing and dereferencing in the code

A Few String Functions

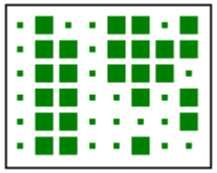


- C string library has useful string functions
  - Rely on null-terminated character arrays

```
#include <string.h>
```

- `strlen(char *s)`: get the length of the array, not counting the null terminator
- `strcpy(char *s, char *t)`: copy array t into s *(make sure the s array is large enough!)*
- `strcmp(char *s, char *t)`: lexicographically compare arrays; 0 if equal, <0 if `s` before `t`, and >0 if `s` after `t` (case sensitive!)

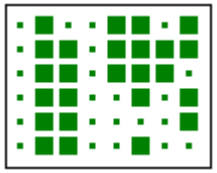
Try a Few String Functions



- We'll use these string library functions and see the results
- File: **stringtest.c**

```
#include <stdio.h>
#include <string.h>
/* Try out a few of the string library functions */
int main() {
        char *a = "parser";
        char *b = "parsley";
        char c[20];
        strcpy(c, a);
        printf("a:%s len:%d\n", a, strlen(a));
        printf("b:%s len:%d\n", b, strlen(b));
        printf("c:%s len:%d\n", c, strlen(c));
        printf("strcmp %s vs. %s: %d\n", a, b, strcmp(a, b));
        printf("strcmp %s vs. %s: %d\n", a, c, strcmp(a, c));
}
```

Homework



- Wednesday (10/14)
  - **Lab** 3 due (scanner, lexical analyzer)
- Due before Friday's class (10/16)
  - LLM: Developer survey... Perusall
- Due Sunday night (10/18)
  - **Journal** entry for week 6
    - 2-1 rule (minimum 2 paragraphs, 1 question)
- Due before Monday's class (10/19)
  - **Read** Chapter 5 Perusall